

Software Development

Módulo: Design Patterns

MOVIE MANAGEMENT

Projeto

Módulo: Design Patterns

Formadora: Sara Maria Andrade Formoso

Formanda: Joana Luísa Pinheiro de Oliveira

08 de junho de 2026

Questões de Reflexão

1. Porque é importante separar responsabilidades pelas camadas?

A separação por camadas permite organizar melhor o código e atribuir responsabilidades específicas a cada parte da aplicação. Desta forma, a interface com o utilizador, as regras de negócio, o acesso aos dados e as entidades ficam independentes entre si. Esta abordagem facilita a manutenção, a evolução do sistema e a correção de erros, tornando o código mais organizado e reutilizável.

2. Qual a vantagem das interfaces?

As interfaces permitem definir contratos que devem ser cumpridos pelas classes que as implementam. Isto reduz o acoplamento entre componentes e aumenta a flexibilidade da aplicação. No projeto, os *Services* dependem das interfaces dos *Repositories* e não das suas implementações concretas, o que permitiu substituir facilmente os *Repositories* em memória por *Repositories SQLite* sem alterar as regras de negócio.

3. O que mudou ao introduzir SQLite?

Inicialmente os dados eram armazenados apenas em memória através de listas, sendo perdidos sempre que a aplicação era encerrada.

Com a introdução do SQLite, os dados passaram a ser guardados numa base de dados persistente, permitindo manter categorias, realizadores e filmes entre execuções da aplicação. Foi também necessário implementar novos *Repositories* responsáveis pelo acesso à base de dados.

4. Porque é importante validar relações entre entidades?

A validação das relações garante a consistência dos dados.

No projeto, um filme só pode ser criado se existir pelo menos uma categoria e um realizador registados. Além disso, é necessário validar se os identificadores escolhidos correspondem a entidades existentes. Desta forma, evitam-se referências inválidas e dados incoerentes na aplicação.

5. Que vantagens trouxe a arquitetura utilizada?

A arquitetura em camadas trouxe uma melhor organização do código, separação de responsabilidades e maior facilidade de manutenção. Permitiu implementar regras de negócio nos *Services*, concentrar o acesso aos dados nos *Repositories* e manter a interface do utilizador independente da forma como os dados são armazenados. Esta estrutura também facilitou a evolução do projeto e a introdução do *SQLite*.

6. Que melhorias poderiam ser feitas no código?

Como melhorias futuras, poderiam ser adicionadas funcionalidades de edição de filmes, categorias e realizadores, uma vez que atualmente apenas é possível adicionar, listar, procurar e remover registos. Também seria possível melhorar a validação dos dados introduzidos pelo utilizador, evitando erros de introdução e tornando a aplicação mais robusta.

Outra melhoria seria otimizar algumas pesquisas e tornar a interface mais intuitiva e amigável para o utilizador.

Conclusão

A realização deste projeto permitiu aplicar na prática conceitos fundamentais de *Design Patterns* e Arquitetura em Camadas. Ao longo do desenvolvimento foram implementadas entidades, interfaces, *services*, *repositories* e mecanismos de persistência de dados utilizando *SQLite*.

A evolução do projeto demonstrou a importância da separação de responsabilidades, da utilização de interfaces para reduzir o acoplamento entre componentes e da validação das regras de negócio para garantir a consistência dos dados.

Com a implementação dos relacionamentos entre Filmes, Categorias e Realizadores e a introdução da persistência *SQLite*, foi possível construir uma aplicação mais completa, organizada e próxima de um cenário real de desenvolvimento de *software*.

Este projeto contribuiu para consolidar conhecimentos de programação orientada a objetos, arquitetura de software, persistência de dados e boas práticas de desenvolvimento em C#.